CTF Writeup – Esposizione Repository Git (Werkzeug)

Nov 5, 2026 · @Lorenzo

Introduzione

Questo report documenta la risoluzione di una macchina CTF basata su un'applicazione web Python servita da **Werkzeug** (il server di sviluppo di Flask). L'obiettivo era catturare una flag nel formato `THM{...}`.

A differenza di un classico attacco applicativo, la catena di sfruttamento qui si basa su un errore di configurazione infrastrutturale: l'esposizione pubblica della cartella `.git/` del progetto, che ha permesso di ricostruire l'intero repository sorgente e recuperare dati non più presenti nella versione live dell'applicazione.

Fase	Tecnica	Risultato
Ricognizione	Nmap	Porta 8080, Werkzeug httpd 3.0.1 (Python 3.12.3)
Ricerca vulnerabilità	CVE-2024-34069, Metasploit	RCE nel debugger individuata ma non sfruttabile
Enumerazione	Gobuster	Nessun risultato diretto
Scoperta	Indizio della sfida + test manuale	Cartella `.git/` esposta pubblicamente
Exploitation	git-dumper, wget mirror	Dump completo del repository
Post-exploitation	git checkout, grep ricorsivo	Codice sorgente ricostruito, flag recuperata

Ricognizione

La scansione iniziale con Nmap individua un solo servizio esposto:

```
PORT     STATE SERVICE
8080/tcp open  http    Werkzeug httpd 3.0.1 (Python 3.12.3)
```

Werkzeug è il server WSGI di sviluppo utilizzato di default da Flask. Non essendoci risultati utili su Exploit-DB, la ricerca si sposta su Google, dove emerge una vulnerabilità nota:

> **CVE-2024-34069** – Esecuzione Remota di Codice (RCE) nel Debugger — Gravità: Alta (CVSS 7.5)

La vulnerabilità viene verificata anche su Metasploit, dove risulta disponibile un modulo dedicato:

```
exploit/multi/http/werkzeug_debug_rce
```

Nonostante diversi tentativi, lo sfruttamento di questa CVE non va a buon fine: il debugger interattivo di Werkzeug non risulta accessibile o sfruttabile in questa configurazione. È necessario quindi cercare un vettore di attacco alternativo.

Enumerazione

Viene avviata un'enumerazione delle directory con Gobuster, inizialmente con la wordlist `small`, senza alcun risultato. Il tentativo viene ripetuto con una wordlist più estesa includendo estensioni come `.zip` e `.bak`, ma anche questa strada non produce risultati utili.

Rileggendo la descrizione della sfida, l'itinerario suggerito fornisce un indizio chiave:



L'obiettivo indicato — *"Dump the exposed source code"* — suggerisce esplicitamente la presenza di codice sorgente esposto da qualche parte sul server, orientando la ricerca verso un endpoint non coperto dall'enumerazione standard con Gobuster.

Scoperta dell'Esposizione della Cartella .git

Seguendo l'indizio, viene testato manualmente l'endpoint `/.git/`, che risulta accessibile e restituisce un directory listing completo:



Questo conferma che l'intera cartella `.git` del progetto — normalmente presente solo in locale durante lo sviluppo — è stata erroneamente pubblicata insieme al codice dell'applicazione, rendendo l'intero storico del repository potenzialmente recuperabile.

Accedendo al file `HEAD` si trova il riferimento al branch attivo:

```
ref: refs/heads/main
```

Seguendo questo riferimento fino a `/.git/refs/heads/main`, viene recuperato l'hash dell'ultimo commit:

```
0f13550b4cb13e9f30c61d5b342c532d21e45bda
```

Con l'hash del commit disponibile, è ora possibile procedere al recupero completo del repository.

Dump del Repository Git

Per enumerare ed estrarre in modo automatico tutti gli oggetti del repository esposto è stato utilizzato **git-dumper**, uno strumento specifico per questo tipo di scenario, che individua ed effettua il download sistematico degli endpoint `.git/` raggiungibili.

In alternativa/parallelo, è stato utilizzato anche `wget` in modalità mirror per scaricare ricorsivamente l'intera struttura:

```
wget --mirror --no-parent -P recovered http://<target>:8080/.git/
```

Il comando salva tutti i file recuperati all'interno della cartella `recovered/<target>:8080/.git/`, replicando fedelmente la struttura originale del sito e dell'endpoint esposto.

Ricostruzione del Codice Sorgente

I file scaricati rappresentano gli oggetti Git grezzi (blob, commit, albero), non ancora leggibili come normali file sorgente. Per "srotolarli" nei file originali è stato eseguito:

```
git checkout -- .
```

Questo comando ricostruisce i blob e i commit presenti negli oggetti `.git` nei relativi file sorgente, riportando alla luce il codice e i file del progetto così come erano stati versionati — inclusi eventuali file sensibili o di configurazione non più presenti nella versione live del sito, ma rimasti nella storia del repository.

Cattura della Flag

Dopo essere tornato nella directory di lavoro iniziale (`cd -`), è stata effettuata una ricerca ricorsiva nel codice recuperato per individuare la flag, riconoscibile dal formato `THM{...}`:

```
grep -RIn 'THM{' recovered
```

La ricerca va a buon fine e individua la flag all'interno del file `README.md`:

```
recovered/README.md:6:Staging flag (remove before launch): THM{byt3_l0tus_n3v3r_f0rg3ts}
recovered/10.130.162.96:8080/README.md:6:Staging flag (remove before launch): THM{byt3_l0tus_n3v3r_f0rg3ts}
```

La flag si trovava in un commento del tipo *"Staging flag (remove before launch)"*, segno di un file di staging/debug dimenticato nella history del repository e mai rimosso prima del deploy in produzione.

Conclusioni

Riepilogo della catena di attacco

1. **Ricognizione** – Nmap individua un server Werkzeug (Flask dev server) sulla porta 8080.
2. **Ricerca vulnerabilità** – Viene identificata CVE-2024-34069 (RCE nel debugger), verificata ma non sfruttabile in questo scenario.
3. **Enumerazione** – Gobuster non restituisce risultati utili; l'indizio della sfida orienta verso la ricerca di codice sorgente esposto.

4. **Scoperta** – L'endpoint `/.git/` risulta pubblicamente accessibile, esponendo l'intero repository Git del progetto.
5. **Exploitation** – Il repository viene scaricato interamente con git-dumper e `wget --mirror`.
6. **Ricostruzione** – `git checkout` riporta alla luce i file sorgente originali dagli oggetti Git.
7. **Cattura flag** – Una ricerca con `grep` individua la flag dimenticata in un commento di staging nel `README.md`.

Lezione principale

Lo sfruttamento dell'esposizione della cartella `.git/` ha permesso di ricostruire il repository sorgente del target e recuperare informazioni — inclusa la flag — che non sarebbero state altrimenti accessibili dalla sola interfaccia web applicativa. Questo scenario evidenzia un errore di configurazione molto comune e ad alto impatto: il deploy di applicazioni web senza escludere le cartelle di controllo versione (`.git/`, `.svn/`, ecc.) dalla root pubblica del webserver, che può esporre credenziali, segreti e l'intera storia di sviluppo del codice.